

Implement a stack using singly linked list

Last Updated : 20 Mar, 2025

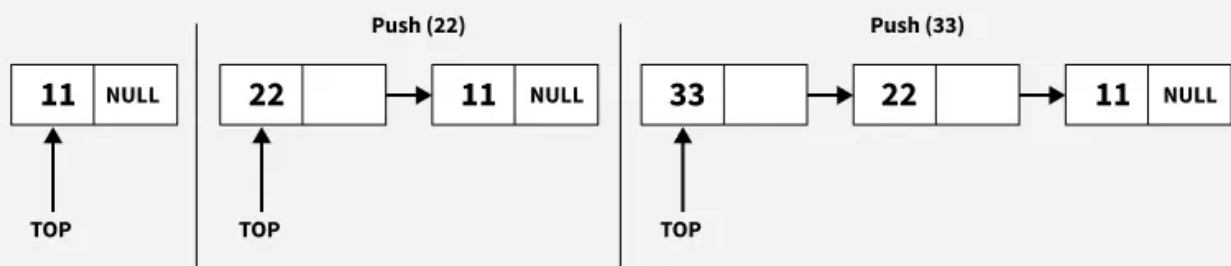


To implement a stack using a singly linked list, we need to ensure that all operations follow the **LIFO** (Last In, First Out) principle. This means that the most recently added element is always the first one to be removed. In this approach, we use a singly linked list, where each node contains data and a reference (or link) to the next node.

To manage the stack, we maintain a top pointer that always points to the most recent (topmost) node in the stack. The key stack operations—push, pop, and peek can be performed using this top pointer.

01
Step

Push elements in the stack

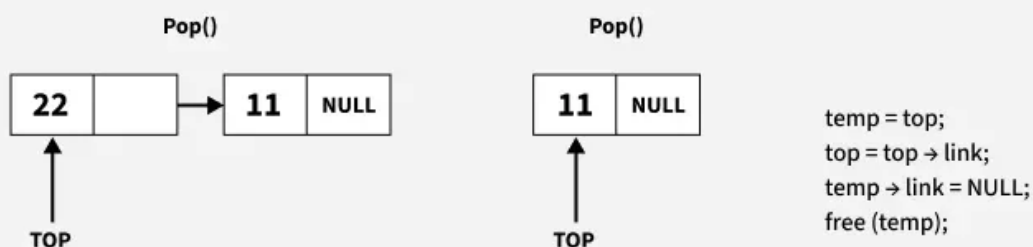


To push a new element onto the stack, create a temporary node temp. Assign the data value and link the temp node to the current top by setting temp->link = top. Finally, update the top pointer to point to the newly created node by setting top = temp.

Implement a stack using singly linked list

02
Step

Pop elements from the stack



```
temp = top;
top = top->link;
temp->link = NULL;
free (temp);
```

Implement a stack using singly linked list

Implement a stack using singly linked list

Last Updated : 20 Mar, 2025



In the stack Implementation, a stack contains a top pointer. which is the "head" of the stack where pushing and popping items happens at the head of the list. The first node has a null in the link field and second node-link has the first node address in the link field and so on and the last node address is in the "top" pointer.

The main advantage of using a linked list over arrays is that it is possible to implement a stack that can shrink or grow as much as needed. Using an array will put a restriction on the maximum capacity of the array which can lead to stack overflow. Here each new node will be dynamically allocated. so overflow is not possible.

Stack Operations

- **push():** Insert a new element into the stack (i.e just insert a new element at the beginning of the linked list.)
- **pop():** Return the top element of the Stack (i.e simply delete the first element from the linked list.)
- **peek():** Return the top element.
- **display():** Print all elements in Stack.

Push Operation

- *Initialise a node*
- *Update the value of that node by data i.e. **node->data = data***
- *Now link this node to the top of the linked list*
- *And update top pointer to the current node*

Pop Operation

- *First Check whether there is any node present in the linked list or not, if not then return*
- *Otherwise make pointer let say **temp** to the top node and move forward the top node by 1 step*
- *Now free this temp node*

Peek Operation

- *Check if there is any node present or not, if not then return.*
- *Otherwise return the value of top node of the linked list*

Display Operation

- *Take a **temp** node and initialize it with top pointer*
- *Now start traversing temp till it encounters NULL*

- *Simultaneously print the value of the temp node*

C++

C

Java

Python

C#

JavaScript

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int new_data) {
        this->data = new_data;
        this->next = nullptr;
    }
};

class Stack {
    Node* head;

public:
    Stack() { this->head = nullptr; }

    bool isEmpty() {
        return head == nullptr;
    }

    void push(int new_data) {
        Node* new_node = new Node(new_data);
        if (!new_node) {
            cout << "\nStack Overflow";
        }
        new_node->next = head;
        head = new_node;
    }

    void pop() {
        if (this->isEmpty()) {
            cout << "\nStack Underflow" << endl;
        } else {
            Node* temp = head;
```

```

        head = head->next;
        delete temp;
    }
}

int peek() {
    if (!isEmpty())
        return head->data;
    else {
        cout << "\nStack is empty";
        return INT_MIN;
    }
}

};

int main() {
    Stack st;

    st.push(11);
    st.push(22);
    st.push(33);
    st.push(44);

    cout << "Top element is " << st.peek() << endl;

    cout << "Removing two elements..." << endl;
    st.pop();
    st.pop();

    cout << "Top element is " << st.peek() << endl;

    return 0;
}

```

Output

```

Top element is 44
Removing two elements...
Top element is 22

```

Time Complexity: $O(1)$, for all `push()`, `pop()`, and `peek()`, as we are not performing any kind of traversal over the list.

Auxiliary Space: $O(n)$, where n is the size of the stack

Benefits of implementing a stack using a singly linked list

- **Dynamic memory allocation:** The size of the stack can be increased or decreased dynamically by adding or removing nodes from the linked list, without the need to allocate a fixed amount of memory for the stack upfront.
- **Efficient memory usage:** Since nodes in a singly linked list only have a next pointer and not a prev pointer, they use less memory than nodes in a doubly linked list.
- **Easy implementation:** Implementing a stack using a singly linked list is straightforward and can be done using just a few lines of code.
- **Versatile:** Singly linked lists can be used to implement other data structures such as queues, linked lists, and trees.

Real time examples of stack

Stacks are used in various real-world scenarios where a last-in, first-out (LIFO) data structure is required. Here are some examples of real-time applications of stacks:

- **Function Call Stack:** When a function is called, its return address and parameters are pushed onto the stack. The stack ensures functions execute and return in reverse order..
- **Undo/Redo Operations:** In apps like text or image editors, actions are pushed onto a stack. Undo removes the last action, while redo restores it.
- **Browser History:** Browsers use stacks to track visited pages. Visiting a page pushes its URL onto the stack, and the "Back" button pops the last URL to go to the previous page.
- **Expression Evaluation:** In compilers, expressions are converted to postfix notation and evaluated using a stack.
- **Call Stack in Recursion:** Recursive function calls are pushed onto the stack. Once recursion ends, the stack is popped to return to the previous function call.

[Comment](#)[More info ▼](#)[Advertise with us](#)[Next Article >](#)

Introduction to Monotonic Stack - Data Structure and Algorithm Tutorials

Similar Reads

Stack Data Structure

A Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO (Last In First Out) or FILO (First In Last Out). LIFO implies that the element that is...

🕒 3 min read

What is Stack Data Structure? A Complete Tutorial

Stack is a linear data structure that follows LIFO (Last In First Out) Principle, the last element inserted is the first to be popped out. It means both insertion and deletion operations happen at one end only....

🕒 4 min read

Applications, Advantages and Disadvantages of Stack

A stack is a linear data structure in which the insertion of a new element and removal of an existing element takes place at the same end represented as the top of the stack. Stack Data...

🕒 2 min read

Implement a stack using singly linked list

To implement a stack using a singly linked list, we need to ensure that all operations follow the LIFO (Last In, First Out) principle. This means that the most recently added element is always the first one to...

🕒 13 min read

Introduction to Monotonic Stack - Data Structure and Algorithm Tutorials

A monotonic stack is a special data structure used in algorithmic problem-solving. Monotonic Stack maintaining elements in either increasing or decreasing order. It is commonly used to efficiently solve...

🕒 12 min read

Difference Between Stack and Queue Data Structures

In computer science, data structures are fundamental concepts that are crucial for organizing and storing data efficiently. Among the various data structures, stacks and queues are two of the most basic yet...

Stack implementation in different language



Some questions related to Stack implementation



Easy problems on Stack



Intermediate problems on Stack

